

作业7

姓名：王泽黎 学号：2022K8009929011

7.1

由题可知进程 T1, T2 中共有四个原子操作 V(S1), V(S2), P(S1), P(S2), 且信号量 S1, S2 的初值为0。则可能情况如下：

(1)

T1 执行 $y:=1$; $y:=y+2$; V(S1); $z:=y+1$; 在执行 P(S2) 时等待, T2 执行 $x:=1$; $x:=x+1$; P(S1); $x:=x+y$; V(S2); T1 的 P(S2) 结束等待, T1 执行 $y:=z+y$; T2 执行 $z:=x+z$;
这种情况下 $x=5$, $y=7$, $z=9$ 。

(2)

T1 执行 $y:=1$; $y:=y+2$; V(S1); $z:=y+1$; 在执行 P(S2) 时等待, T2 执行 $x:=1$; $x:=x+1$; P(S1); $x:=x+y$; V(S2); $z:=x+z$; T1 的 P(S2) 结束等待, T1 执行 $y:=z+y$;
这种情况下 $x=5$, $y=12$, $z=9$ 。

(3)

T1 执行 $y:=1$; $y:=y+2$; V(S1); T2 执行 $x:=1$; $x:=x+1$; P(S1); $x:=x+y$; V(S2); $z:=x+z$; T1 执行 $z:=y+1$; P(S2); T1 执行 $y:=z+y$;
这种情况下 $x=5$, $y=7$, $z=4$ 。

7.2

方法一和方法二的主要区别在于 Deposit(c) 与 Remove(c) 中信号量相关操作和锁相关操作的执行顺序不同, 进而导致方法一可以让生产者、消费者进程正常运行, 而方法二可能导致生产者、消费者进程死锁。

例如: 当使用方法二时, 如果缓冲区空时, 消费者操作缓冲区, 消费者先取锁, 然后消费者开始等待生产者, 但生产者无法取得锁, 导致死锁。

7.3

```
Class BankBuffer{  
    mutex = new Semaphore(1);  
    fullBuffer = new Semaphore(0);
```

```
emptyBuffer = new Semaphore(n);  
}
```

```
BankBuffer :: Customer_Service{  
emptyBuffer.P();  
mutex.P();  
Customer gets number;  
mutex.V();  
fullBuffer.V();  
}
```

```
BankBuffer :: Teller_Service{  
fullBuffer.P();  
mutex.P();  
Teller calls number;  
mutex.V();  
emptyBuffer.V();  
}
```

7.4

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define NUM_THREADS 8
#define NUM_TASKS 16

typedef struct {
    int tasks[NUM_TASKS];
    int task_index;
    int results[NUM_TASKS / 2];
    int result_index;
    pthread_mutex_t mutex;
    pthread_cond_t cond;
} Monitor;

Monitor monitor;

void init_monitor(Monitor *m) {
    srand(time(NULL));
    for (int i = 0; i < NUM_TASKS; i++) {
        m->tasks[i] = rand() % 100;
    }
    m->task_index = 0;
    m->result_index = 0;
    pthread_mutex_init(&m->mutex, NULL);
    pthread_cond_init(&m->cond, NULL);
}

void getTask(Monitor *m, int *a, int *b) {
    pthread_mutex_lock(&m->mutex);
    while (m->task_index >= NUM_TASKS) {
        pthread_cond_wait(&m->cond, &m->mutex);
    }
    *a = m->tasks[m->task_index++];
    *b = m->tasks[m->task_index++];
    pthread_mutex_unlock(&m->mutex);
}
```

```

void putResult(Monitor *m, int result) {
    pthread_mutex_lock(&m->mutex);
    m->results[m->result_index++] = result;
    pthread_cond_signal(&m->cond);
    pthread_mutex_unlock(&m->mutex);
}

void *thread_func(void *arg) {
    int a, b, result;
    while (1) {
        getTask(&monitor, &a, &b);
        result = a + b;
        usleep((rand() % 10 + 1) * 1000); // 模拟随机延迟
        putResult(&monitor, result);
        if (monitor.task_index >= NUM_TASKS) break;
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    init_monitor(&monitor);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, thread_func, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Result:\n");
    for (int i = 0; i < monitor.result_index; i++) {
        printf("%d ", monitor.results[i]);
    }
    printf("\n");

    pthread_mutex_destroy(&monitor.mutex);
    pthread_cond_destroy(&monitor.cond);

    return 0;
}

```